University M'hamed Bougara of Boumerdes
Faculty of Sciences – Department of Computer Science
Lab Data Mining, 4th Year Engineer

Lab N° 1

1. Lists (delimited by [ ])
2. List Comprehensions
3. Tuples (delimited by ( ))
4. Sets (delimited by { })
5. Dictionary
6. Functions
7. File Management

1. Lists (delimited by [ ])

- *List is an ordered sequence of items.*
- *We can have different data types under a list. E.g we can have integer, float and string items in a same list. We can have a list of lists.*

Exercise 1. Consider the following list of items.

mylist = ['one' , 'two' , 'three' , 'four' , 'five' , 'six' , 'seven' , 'eight'].

1. Write a Python Program, using predefined functions related to Lists, to:
- Return all items from 0th to 3rd index location.
- List all items from 2nd to 5th index location.

2. Write a Python Program, using predefined functions related to Lists to:
- Add an item to the end of the list.
- Add item at index location 9.
- Remove last item of the list.
- Remove item at index location 8, using two different methods.
- Reverse the list.
- Number of times item "two" occurred in the list.
- Empty List / Delete all items in the list.

3. Create a copy of mylist, mylist1 using two different methods: assignment and copy() function. What do you observe in terms of the address of both mylist & mylist1?

Exercise 2. Consider the following list of items. mylist3 = [9, 5, 2, 99, 12, 88, 34]. Write a Python Program, using predefined functions related to Lists, to Sort mylist3 in ascending order, then in descending order.

2. List Comprehensions

- *List Comprehensions provide an elegant way to create new lists.*
- *It consists of brackets containing an expression followed by a for clause, then zero or more if clauses.*

[*Expression for item in List*]
[*x**2 for x in list range (10)*]

Example:

```
In [287] : mystring = "WELCOME"
           mylist = [ i for i in mystring ] # iterating through a string using List Comprehension
Out[287] : [ 'W', 'E', 'L', 'C', 'O', 'M', 'E' ]
```

Exercise 3. Using List Comprehension.

- Create and Display all even numbers between 0 and 40.
- Calculate square of all numbers between.
- Multiple whole list of numbers by 10.
- List all numbers divisible by 3 , 9 & 12 using nested "if" .
- What does this line of code :
  l1 = [print("{} is Even Number".format(i)) **if** i%2==0 **else** print("{} is odd number"].

3. Tuples (delimited by ( ))

- *Tuple is similar to List except that the objects in tuple are immutable which means we cannot change the elements of a tuple once assigned.*
- *When we do not want to change the data over time, tuple is a preferred data type.*
- *Iterating over the elements of a tuple is faster compared to iterating over a list.*

Exercise 4. Using List Comprehension.

- Create Tuple of mixed data types.
- Create nested Tuple of mixed data types.
- *Note : Tuple Indexing and Tuple slicing is similar as for lists.*

4. Sets (delimited by { })

- *Unordered & Unindexed collection of items.*
- *Set elements are unique. Duplicate elements are not allowed.*
- *Set elements are immutable (cannot be changed).*
- *Set itself is mutable. We can add or remove items from it.*

Exercise 5.

- Create a Set containing the numbers from 1 to 5.
- Create a Set of mixed data types: numbers, strings, tuple.
- Create a Set of mixed data types: numbers, strings, list.
- Convert the ('one' , 'two' , 'three' , 'four') to a set.

5. Dictionary

- *Dictionary is a mutable data type in Python.*
- *A python dictionary is a collection of key and value pairs separated by a colon (:) & enclosed in curly braces {}.*
- *Keys must be unique in a dictionary, duplicate values are allowed.*

mydict = {'A': 'one' , 'B': 'two' , 'C': 'three'}. 'A','B','C' are Keys and 'one', 'two', 'three' are values.

Exercise 6.

1. **Create:** - An empty dictionary using two different methods.
   - A dictionary from a sequence of keys.
   - A dictionary from a sequence of keys and value.
2. **Consider the following dictionary**

mydict1 = {'Name': 'Salah' , 'ID': 2022035 , 'DOB': 2003 , 'Address' : 'Algiers'}

- Change Dictionary Items : DOB and Adress.
- Add 'Job' item in mydict1.
- Remove item "address" in the dictionary using *Pop* and *del* methods.
- Create a new reference "mydict1" and a copy "mydict2". What is the difference between the two.

3. **Consider mydict1 again**
   - Disply Key & value pair using for loop.
   - Test if the key "Name" is in mydict1 or not.

4. **Dictionary Comprehension**

   - Double each value between 0 and 10 using dictionary comprehension.
   - Using dictionary comprehension, create a dictionary from key = ['one' , 'two' , 'three' , 'four' , 'five'] and value = [1,2,3,4,5].

6. Functions

- *Parameters are specified after the function name inside the parentheses.*
- *Information can be passed into a function as arguments.*
- *Parameters must be separated with a comma.*
- *A function may or may not return data. In Python a function is defined using the **def** keyword.*

Question 1. How many types of functions exist in Python ? **Answer : 3.**
Syntax and example:

```
def square (n):
    #function to find square of a number
    n= n*n
  return n
```

Exercise 7.

- Write a function to check whether a number is even or odd.
- What is the output of the following of code :

```
var1 = 100
def myfunc():
  var1 = 99
  print(var1)
```

Exercise 8. Calculate factorial of a number using a recursive function.

7.File Management

Python has several built-in modules and functions for creating, reading, updating and deleting files.

Open File ----------

```
In [69]: fileobj = open('test1.txt') # Open file in read/text mode
In [70]: fileobj = open('test1.txt', 'r') # Open file in read mode
In [71]: fileobj = open('test1.txt', 'w') # Open file in write mode
In [72]: fileobj = open('test1.txt', 'a') # Open file in append mode
```

Close File

```
In [73]: fileobj.close()
```

Read File ----------

```
In [84]: fileobj = open('test1.txt')
In [85]: fileobj.read() #Read whole file
```

Out[85]: 'Python generators are easy way of creating iterators. It generates values one at a time from a given sequence instead of returning the entire sequence at once.\nIt is a special type of function which returns an iterator object.\nIn a generator function, a yield statement is used rather than a return statement.\nThe generator function cannot include the return keyword. If we include it then it will terminate the execution of the function.\nThe difference between yield and return is that once yield returns a value the function is paused and the control is transferred to the caller.Local variables and their states are remembered between successive calls. In case of the return statement value is returned and the execution of the function is terminated.\nMethods like __iter()__ and __next()__ are implemented automatically in generator function.\nSimple

generators can be easily created using generator expressions. Generator expressions create anonymous generator functions like lambda.\nThe syntax for generator expression is similar to that of a list comprehension but the only difference is square brackets are replaced with round parentheses. Also list comprehension produces the entire list while the generator expression produces one item at a time which is more memory efficient than list comprehension.'

```
In [86]: fileobj.read() #File cursor is already at the end of the file
Out[86]: ''

In [87]: fileobj.seek(0) # Bring file cursor to initial position.

In [89]:fileobj.read(16) # Return the first 16 characters of the file

Out[89]:'Python generator'

In [90]: fileobj.tell() # Get the file cursor position
Out[90]: 16

print(fileobj.readline()) # Read first line of a file.

print(fileobj.readline()) # Read second line of a file.

print(fileobj.readline()) # Read third line of a file.
```

Write File ----------

```
fileobj = open('test1.txt', 'a')

fileobj.write('THIS IS THE NEW CONTENT APPENDED IN THE FILE') # Append content

fileobj.close()

fileobj = open('test1.txt')

fileobj.read()

Out[95]:
'Python generators are easy way of creating iterators. It generates values one at a time from a given sequence instead of returning the entire sequence at once.\nIt is a special type of function which returns an iterator object.\nIn a generator function, a yield statement is used rather than a return statement.\nThe generator function cannot include the return keyword. If we include it then it will terminate the execution of the function.\nThe difference between yield and return is that once yield returns a value the function is paused and the control is transferred to the caller.Local variables and their states are remembered between successive calls. In case of the return statement value is returned and the execution of the function is terminated.\nMethods like __iter()__ and __next()__ are implemented automatically in generator function.\nSimple generators can be easily created using generator expressions.
```

Generator expressions create anonymous generator functions like lambda.\nThe syntax for generator expression is similar to that of a list comprehension but the only difference is square brackets are replaced with round parentheses. Also list comprehension produces the entire list while the generator expression produces one item at a time which is more memory efficient than list comprehension.THIS IS THE NEW CONTENT APPENDED IN THE FILE'

```
fileobj = open("test1.txt", "w")

fileobj.write("NEW CONTENT ADDED IN THE FILE. PREVIOUS CONTENT HAS BEEN OVERWRITTEN”

fileobj.close()

fileobj = open('test1.txt')

fileobj.read()
```

Out[96]:'NEW CONTENT ADDED IN THE FILE. PREVIOUS CONTENT HAS BEEN OVERWRITTEN'